School of Computing and Augmented Intelligence
ASU IRA A. FULTON SCHOOLS OF engineering

CSE446 / CSE598
Software Integration and Engineering

Unit 3

Business Process Execution and Integration

Lecture 3-4

Web Data Integration through Caching and Recommendation

Text Sections 5.5 and 11.3.2

Dr. Yinong Chen

Roadmap: Web Caching and Recommendation

- Proactive artificial intelligent services and on demand e-business create ever more computing demands
- How can business maintain affordable and real-time performance?
  - **Web Caching**
    - What is **Web Caching**?
    - Why Web Caching?
    - Basic Web Caching Techniques
    - Advanced Caching: Web Data Caching
    - Implementation of Web Data Caching
  - **Web Recommendation**
    - Web Recommendation System versus Web Caching
    - Social Filtering and Recommendation
    - Legal and Ethics Issues in Caching and Recommendation

On Demand e-business with AI-based real-time reconfigure-ation, resilient, & automated evolution

Web 5.0 with proactive artificial intelligent services and applications*

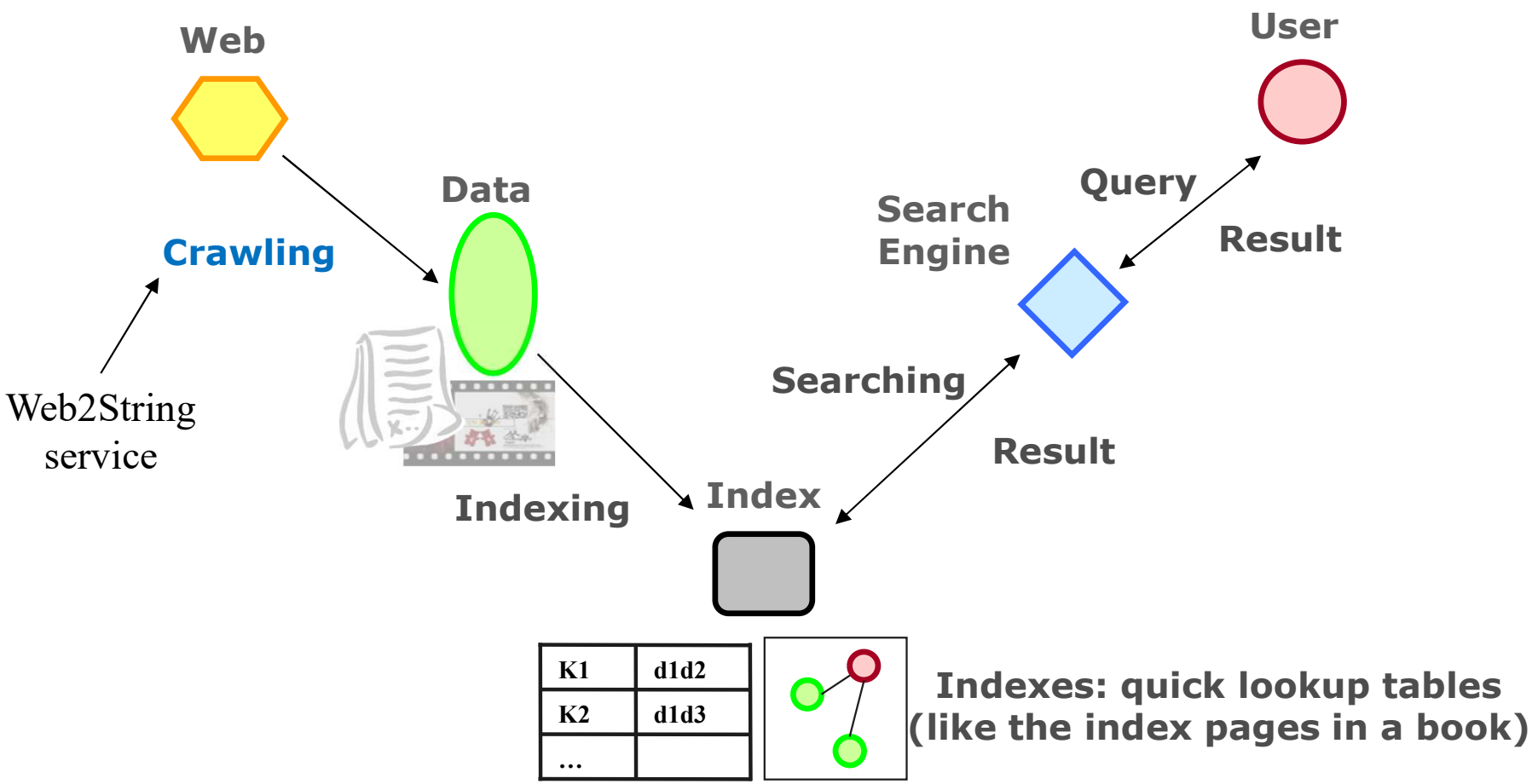
From Memory Caching to Web Caching

- Rationale behind caching in **computer architecture**
  - Temporal locality: Data used are likely to be used again
  - Spatial Locality: Data used, and their neighboring data are likely to be used: Data like array are stored in block
  - Memory accesses are slow
- Reasons for **Web Caching**
  - Web is a super large enterprise system, with a lot of computing, memory & disk accesses, and communications;
  - Data accesses to Web services and databases are slow & costly;
  - Most people want to know the same things, e.g., what other people are reading or buying;
  - A page could continue to exist after the original page becomes unavailable;
  - Supporting data mining and social filtering
- Caching is also expensive: Carefully select caching data and tradeoff between performance gain and cost.

What are Google's

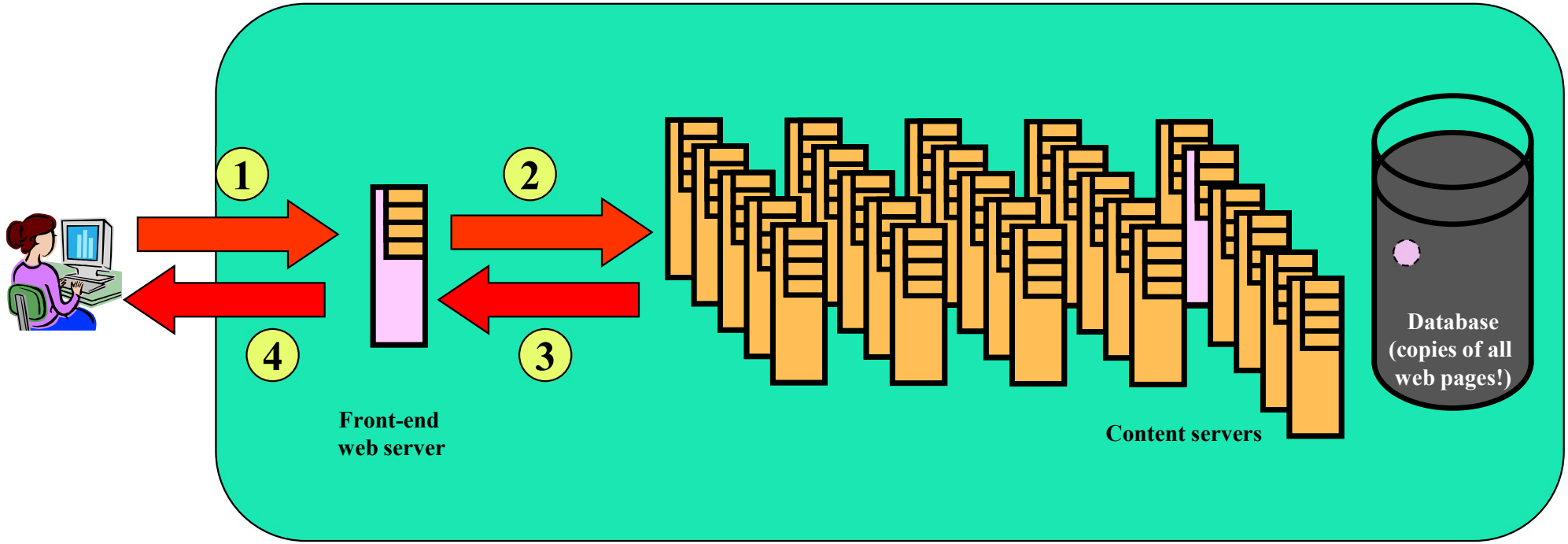
Major Innovations?

(1) Crawling


Web
User
Data
Query
Search
Engine
Result
Crawling
Web2String
service
Searching
Result
Indexing
Index
K1
d1d2
K2
d1d3
...
Indexes: quick lookup tables
(like the index pages in a book)

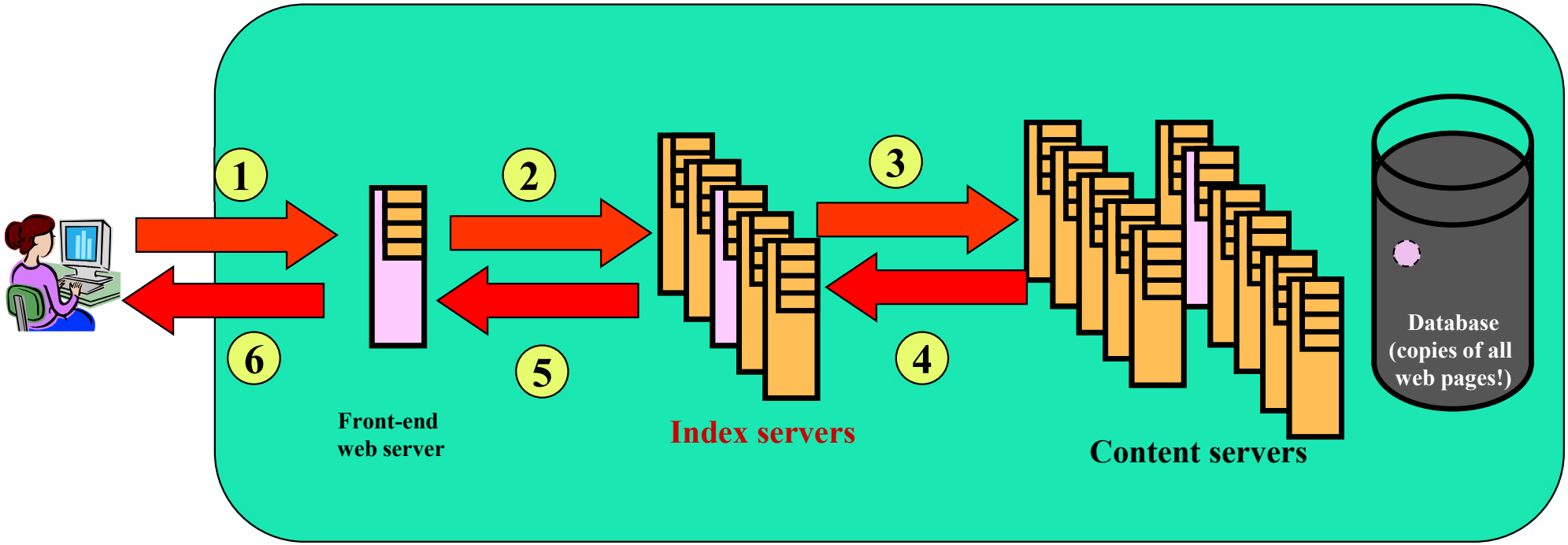
(2) Distributed & Parallel Computing

Web-Based Distributed Software Development



(3) Indexing

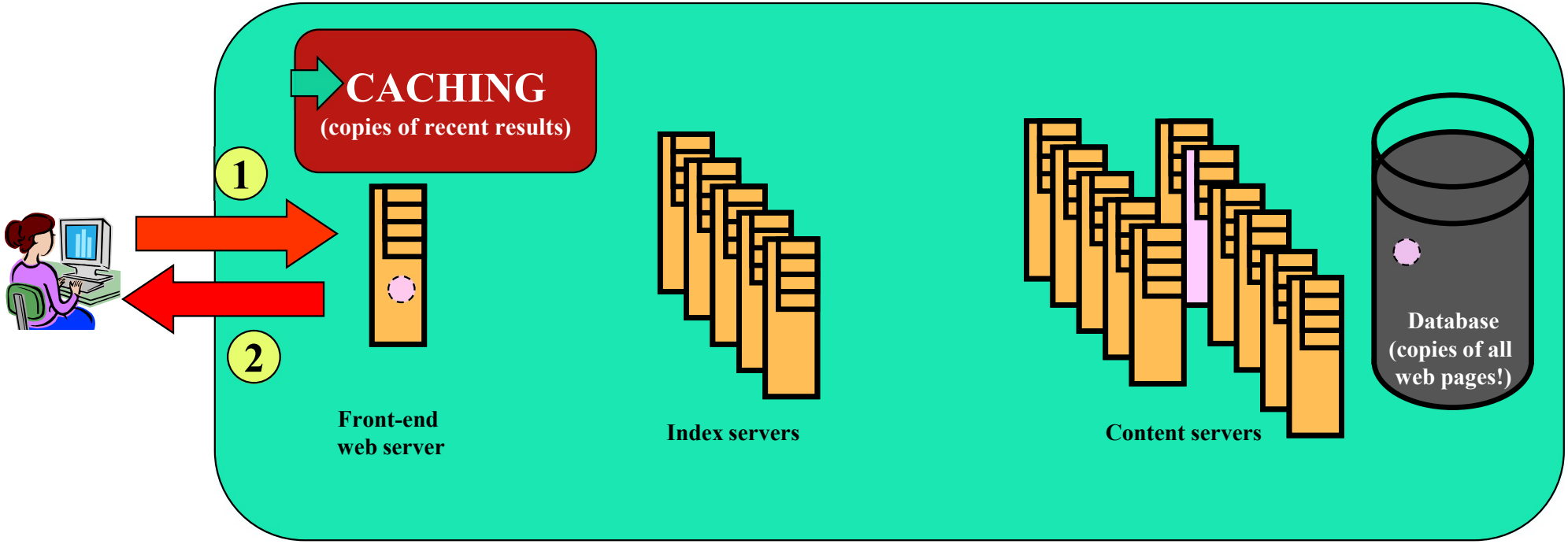
Organize data in layers for fast accesses



- Protocol Buffers for efficient data representation and processing;
- BigTable based on B+ tree for indexing

(4) Web Caching

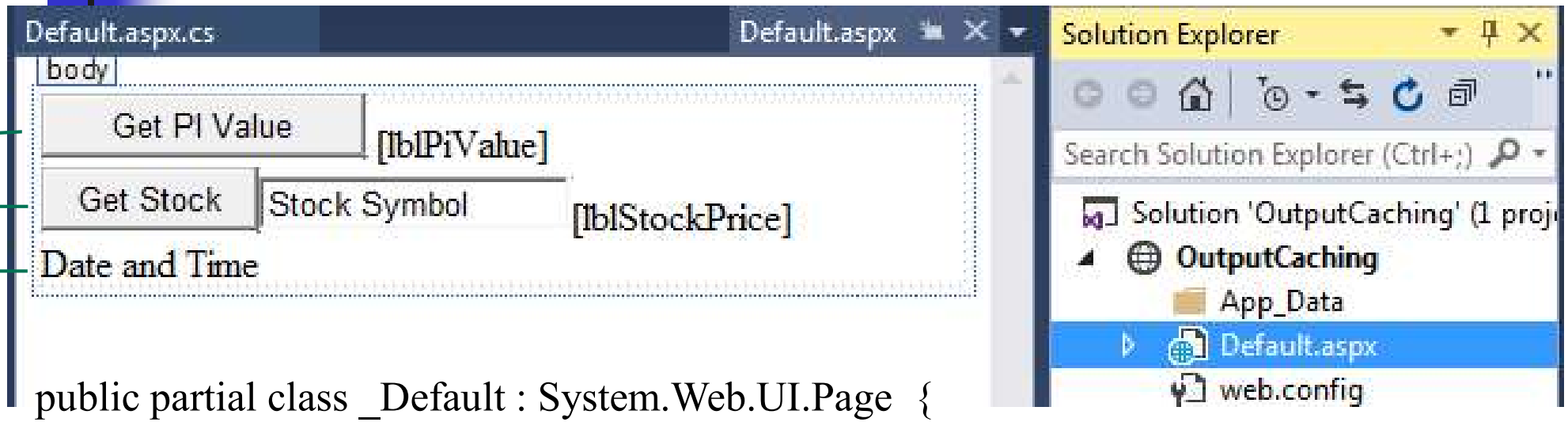
- Most people want to know the same things.
- A page continues to exist after the original page is removed by the creator



Types of Web Caching

Caching Type	Features
Output Caching	The entire xhtml (or html) page generated from aspx page is cached. When the same page is requested, the same rendered page is sent to the browser. Caching duration and location (on server or client side) can be specified. Text 5.5.1
Fragment Caching	Caching the entire page may be too expensive. Specifying a fragment of a page through a **user control** to cache partial data can be a better tradeoff between performance and cost. Text 5.5.2
Data Caching	Selecting individual data objects and organizing them for implementing cross-business logic; Define and program your own data objects for caching, with the support from the runtime environment for threading, placement, and replacement. Read text 5.5.3
Multi-layer Caching	Combine all caching techniques and apply them at all layers: output caching, fragment caching, data caching, and API/service caching

Output Caching: GUI Design




```
public partial class _Default : System.Web.UI.Page  {
    protected void Page_Load(object sender, EventArgs e) {
        String date = DateTime.Now.ToLongDateString();
        String time = DateTime.Now.ToLongTimeString();
        lblDateTime.Text = date + ", " + time;
    }
    protected void btnStock_Click(object sender, EventArgs e) {
        Random r = new Random();  int price = r.Next(100);
        lblStockPrice.Text = Convert.ToString(price);
    protected void btnPiValue_Click(object sender, EventArgs e)  {
        double pi = System.Math.PI;
        lblPiValue.Text = Convert.ToString(pi);
    }
}
```

Should not cache the time

Cache for a few seconds

Or call stock service in ASU WSR

Cache for a long time

Add One Line of Directive to Source Page

```
<%@ Page Language="C#" AutoEventWireup="true"  CodeFile="Default.aspx.cs"
    Inherits="_Default" %>
<%@ OutputCache duration="10" varybyparam="None" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head id="Head1" runat="server">
   <title>Caching Page</title>
</head> <body>  <form id="form1" runat="server">
   <div>
      <asp:Button ID="btnPiValue" runat="server" Text="Get PI Value"  Width="133px"
         onclick="btnPiValue_Click" />  
       <asp:Label ID="lblPiValue" runat="server"></asp:Label>
   </div>
   <asp:Button ID="btnStock" runat="server" onclick="btnStock_Click"
      Text="Get Stock" />
   <asp:TextBox ID="txtAbsoluteValue" runat="server">Stock Symbol</asp:TextBox>
```

Testing of Caching

http://venus.sod.asu.edu/WSRepository/OutputCaching/Default.aspx

Click 3 times within 10 seconds

51 Get PI Value
Get Stock Stock Symbol 95
Tuesday, February 24, 2026, 9:42:51 PM

55 Get PI Value
Get Stock Stock Symbol 95
Tuesday, February 24, 2026, 9:42:51 PM

59 Get PI Value
Get Stock Stock Symbol 95
Tuesday, February 24, 2026, 9:42:51 PM

No value!

Wait longer (> 10 seconds)

29 Get PI Value
Get Stock Stock Symbol 74
Tuesday, February 24, 2026, 9:43:29 PM

Value change

Wait longer (> 10 seconds)

38 Get PI Value
Get Stock Stock Symbol 23
Tuesday, February 24, 2026, 9:45:38 PM

Wait longer (> 10 seconds)

23 Get PI Value 3.14159265358979
Get Stock Stock Symbol 23
Tuesday, February 24, 2026, 9:46:23 PM

Caching with Parameter Values

```
<%@ OutputCache duration="10" varybyparam="None" %>
```

Cache for 10 seconds

Cache one page for all users

varybyparam="Book_Title"

varybyparam="Book_Title; Author"

If parameter values are specified, different pages will be cached for different parameter values

Caching on client side

```
<%@ OutputCache duration="10" varybyparam="partId" Location = "Client" %>
```

What are the pros and cons of client-side caching?

Fragment Caching through User Controls

Please type your message in the text box

Please enter text here

Encrypt Now

Encrypted Message

Send to Receiver

Today's date is [lblDate]

Another reason of using user controls

A User Control in the ASPX page

Add this line in User Control

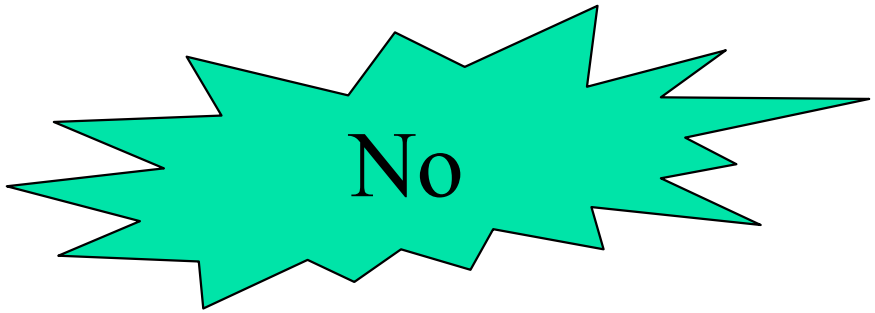
```
<%@ Control Language="C#" AutoEventWireup="true" CodeFile="DateToday.ascx.cs"
Inherits="Confirmation" %>
<%@ OutputCache duration="10" varybyparam="None" %>
<asp:Label ID="lblConformMsg" runat="server" Text="Today's date is "></asp:Label>
<asp:Label ID="lblDate" runat="server" Text=""></asp:Label>
```

Web Data Caching

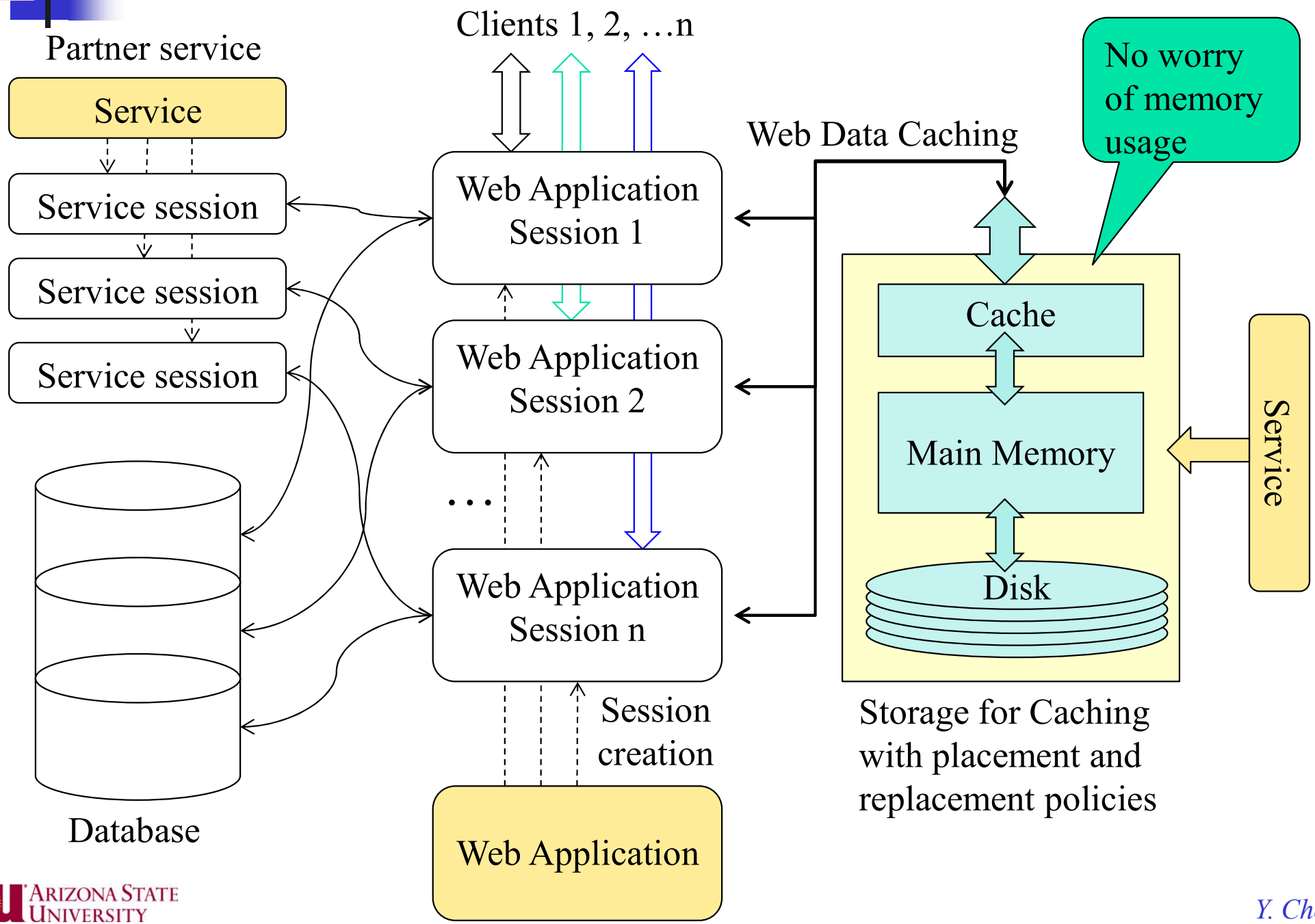
Selecting individual data objects and organizing for implementing unique business logic

Use Application State for Data Caching?

- You could save data objects into Application State and made them available to all sessions.
- What is wrong with using Application State?
  - You have to be aware of how much memory you use;
  - You have to update the lists and dispose the older items and lists;
  - You have to manage the threads and lock the objects while you are updating them.



Organization of Data Caching



ASP .Net Caching Classes

ASP .Net provides a namespace of caching classes that automates the tasks necessary for caching, which allows one to

- **Use the same Application/Session State syntax**: Cache[“IndexKey”] = object.

  Easy to use with a familiar syntax;
- **Automated Expirations** — absolute or relative expirations for underused items to expire automatically.
- **Define dependencies** between cached items and other items
- **Automated Callback** — Event handler can be added that will be called when an item is being removed from the Cache, giving us an opportunity to update the **Cache** or not remove the item.
- **Manage threads** automatically to prevent conflicting.

Namespace: System.Web.Caching in FCL

- Cache class: The class cannot be inherited. When an instance of the class Cache is created in a session, the instance will be
  - available to all sessions, like Application State, and
  - valid till the application is closed or the cache expires.
- CacheDependency class:
  - Establish dependency relationship between an item/object and
    - a file or a database
    - a cache key,
    - an array of either,
    - or another CacheDependency object.
  - The CacheDependency class monitors the dependency relationships so that when a relation changes, the cached item can be automatically removed.

Namespace: System.Web.Caching in FCL

http://msdn.microsoft.com/en-us/library/system.web.caching

- There are many other classes in the namespace, including:
  - AggregateCacheDependency
  - DatabaseNotEnabledForNotificationException
  - FileResponseElement
  - HeaderElement
  - MemoryResponseElement
  - OutputCache
  - OutputCacheProvider
  - OutputCacheProviderCollection
  - ResponseElement
  - SqlCacheDependency
  - SqlCacheDependencyAdmin
  - SubstitutionResponseElement
  - TableNotEnabledForNotificationException

Useful Methods in Cache Class

- **Add**: Adds a specific item to an existing Cache object, such as dependencies, expiration, priority, and a delegate you can use to notify your application when the inserted item is removed from the Cache.
- **Remove**: Removes the specified item from the application's Cache object.
- **Get**: Retrieves the specified item from the Cache object.
- **GetType**: Gets the Type of the current instance. (Inherited from Object)
- **ToString**: Convert the current Object to a string. (Inherited from Object)

Overloaded Insert Methods in Cache Class

- Insert(String, Object), or Cache[“IndexKey”] = Object;
- Insert(String, Object, **CacheDependency**)
- Insert(String, Object, **CacheDependency**, DateTime, TimeSpan)
- Insert(String, Object, **CacheDependency**, DateTime, TimeSpan, CacheItemUpdateCallback)
- Insert(String, Object, **CacheDependency**, DateTime, TimeSpan, CacheItemPriority, CacheItemRemovedCallback)

Parameters of the Insert( ) Methods

```
public void Insert(
    string key,
    Object item,
    CacheDependency dependencies,
    DateTime absoluteExpiration,
    TimeSpan slidingExpiration,
    CacheItemPriority priority,
    CacheItemRemovedCallback
        onRemoveCallback
)
```

- Index key (string)
- Item to be cached
- Dependencies between the cached item and another item.
- The cached item will be removed at the given date and time.
- The cached item will be removed if the item is not accessed within the given time interval. If set to 0, only absolute expiration is used.
- The cost of the object relative to other items stored in the cache.
- A delegate that, if provided, will be called when an object is removed from the cache.

CacheDependency Class

http://msdn.microsoft.com/en-us/library/system.web.caching.cachedependency

- The class defines a parameter of the insert / add method of the Cache class;
- Use null if no need to define the dependency.
- It establishes a dependency relationship between the item stored in the Cache object and a file, a cache key, an array of either, or another CacheDependency object.
- You can include dependency in Insert(), or add it later using Add().
- The CacheDependency class monitors the dependency relationships so that when any of them changes, the cached item can be automatically removed

A simple example of using cache dependency, with one callback

```
protected void btnAddItem_Click(object sender, EventArgs e)
{ // create two cache items
    string item1 = "Dell Printer 1710";
    string item2 = "Cartridge for Printer 1710";
    Cache.Insert("PrinterKey", item1, null, DateTime.Now.AddMinutes(10),
TimeSpan.Zero, CacheItemPriority.Default, CacheRemovedCallBack);
    Cache.Insert("CartridgeKey", item2, null,
DateTime.Now.AddMinutes(20), TimeSpan.Zero);  }
```

Dependency not defined, use an event callback

Delegate with Implicit Parameter Passing. Call when this cache is removed.

Event handler

```
private void CacheRemovedCallBack(string IndexKey, object value,
                                  CacheItemRemovedReason reason)
{ // remove the Cartridge from the cache if printer is removed
    Cache.Remove("CartridgeKey"); // remove cartridge from the
}                                 // cache if printer is removed
```

Simple Cache Example: With Two Callbacks

```
protected void btnAddItem_Click(object sender, EventArgs e)
{   string item1 = "Dell Printer 1710";
    string item2 = "Cartridge for Printer 1710";
    Cache.Insert("PrinterKey", item1, null, DateTime.Now.AddMinutes(10),
TimeSpan.Zero, CacheItemPriority.Default, CacheRemovedCallBack);
    Cache.Insert("CartridgeKey", item2, null,
DateTime.Now.AddMinutes(10), TimeSpan.Zero,
CacheRemovedCallBack);  }
```

Implicit Parameter Passing. Call when this cache is removed.

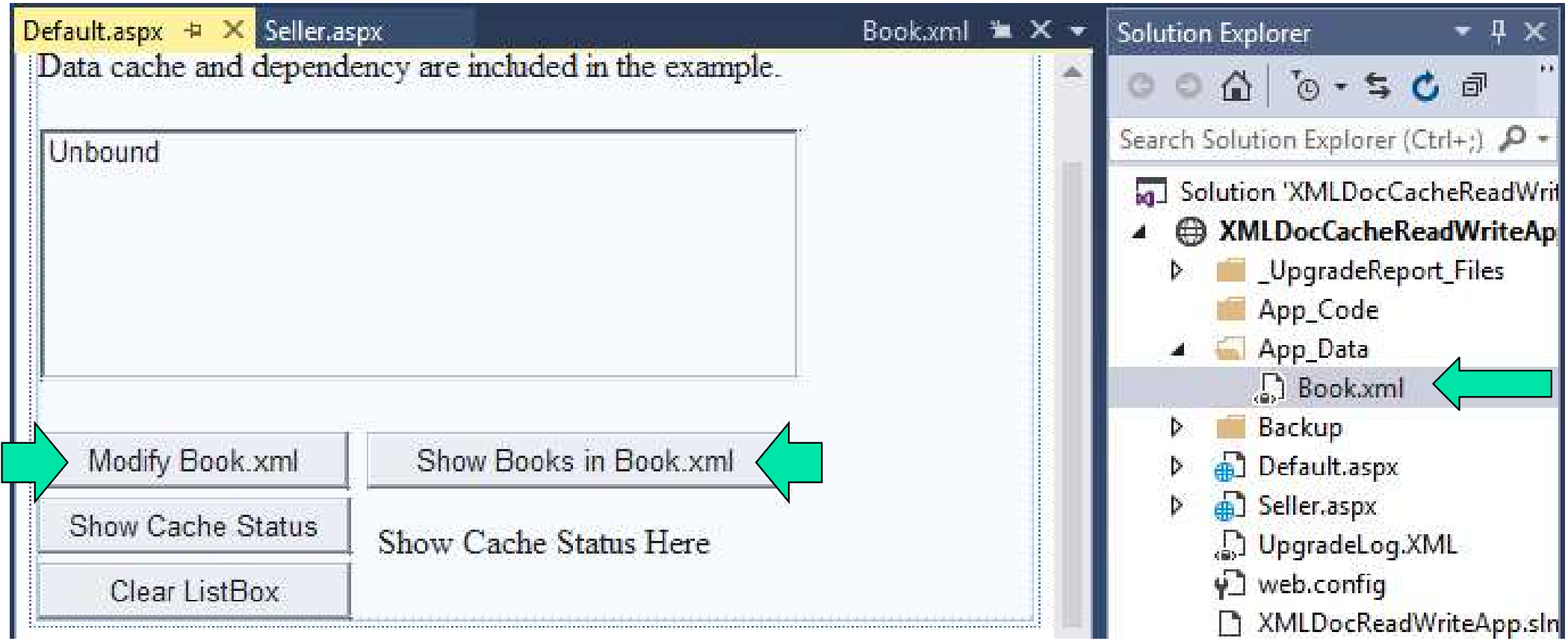
Shared event handler

```
private void CacheRemovedCallBack(string IndexKey, object value,
                                  CacheItemRemovedReason reason)
{ // remove the cartridge from the cache if printer is removed
    if (IndexKey == "PrinterKey" )
        Cache.Remove("CartridgeKey");
    else if (IndexKey == "CartridgeKey")
        Cache.Remove("PrinterKey");  } // Remove printer cache if ...
```

Bookstore Example

http://venus.sod.asu.edu/WSRepository/XMLDocCacheReadWriteApp/Default.aspx

This example is a revised version of the example used for demonstrating the XML and text file operations. The application consists a Default.aspx page and a Seller.aspx page.



Default.aspx: When page is loaded

```
protected void Page_Load(object sender, EventArgs e) {
    if ((Cache["BooksKey"] != null) && (!ckBypassCache.Checked))  {
        ListBox1.Items.Clear();
        XmlDocument xd = (XmlDocument)Cache["BooksKey"];
        XmlNode node = xd;
        XmlNodeList children = node.ChildNodes;
        foreach (XmlNode child in children.Item(1))
        {
            ListBox1.Items.Add(child.FirstChild.InnerText + "  " +
child.FirstChild.NextSibling.InnerText + "  " + "$" + child.LastChild.InnerText);
        }
    }
    if (Cache["BooksKey"] == null)
        lblCacheStatus.Text = "No Cache of Book.xml";
    else
        lblCacheStatus.Text = Cache.ToString();
}
```

If there is data in cache, and cache is not bypassed

Load data from cache. Cache is in memory, not in disk!

Display cache data into ListBox

Display initial cache status

Code Behind Button

```
protected void btnShowCache_Click(object sender, EventArgs e)
  {
      ListBox1.Items.Clear();
      if ((Cache["BooksKey"] != null) && (!ckBypassCache.Checked))
      {
          ListBox1.Items.Clear();
          XmlDocument xd = (XmlDocument)Cache["BooksKey"];
          XmlNode node = xd;
          XmlNodeList children = node.ChildNodes;
          foreach (XmlNode child in children.Item(1))
          {
              ListBox1.Items.Add(child.FirstChild.InnerText + "  " +
child.FirstChild.NextSibling.InnerText + "  " + "$" + child.LastChild.InnerText);
          }
      }
  }
```

The code is similar to the code behind Page_Load

Code Behind the Button

Show Books in Book.xml

```
protected void btbReloadFile_Click(object sender, EventArgs e) {
    FileStream fS = null; string fLocation =
Path.Combine(Request.PhysicalApplicationPath, @"App_Data\Book.xml");
    try { if (File.Exists(fLocation)) {
        fS = new FileStream(fLocation, FileMode.Open, FileAccess.Read);
        XmlDocument xd = new XmlDocument(); xd.Load(fS);
        fS.Close(); // close after loading
        if (Cache["BooksKey"] == null) {  // It was removed when file is changed
            CacheDependency fileDependency = new CacheDependency(fLocation);
            Cache.Insert("BooksKey", xd, fileDependency,
DateTime.Now.AddMinutes(30), TimeSpan.Zero, CacheItemPriority.Default,
                new CacheItemRemovedCallback(CacheRemovedCallBack));
            XmlNode node = xd;
            XmlNodeList children = node.ChildNodes;
            foreach (XmlNode child in children.Item(1)) {
                ListBox1.Items.Add(child.FirstChild.InnerText + "  " +
child.FirstChild.NextSibling.InnerText + "  " + "$" + child.LastChild.InnerText);}}}}
finally{ fS.Close(); }} // close in case the session crashes
```

Add data into cache and define the dependency to the file & callback

Define the Callback Handler

```
private void CacheRemovedCallBack(string IndexKey, object value,
                                  CacheItemRemovedReason reason)
  { // remove the cache if the file is changed
    if (IndexKey == "BooksKey")
      Cache.Remove("BooksKey");
  }
  protected void btnShowCache_Click(object sender, EventArgs e)
  {
    if (Cache["BooksKey"] == null)
      lblCacheStatus.Text = "No Cache of Book.xml";
    else lblCacheStatus.Text = Cache.ToString();
  }
protected void btnClear_Click(object sender, EventArgs e)  { // Clear ListBox
    ListBox1.Items.Clear();
    ListBox1.Items.Add("The ListBox is cleared... ");
  }
}
```

Callback event handler

Remove the key from Cache

Show Cache Status

Show Cache Status Here

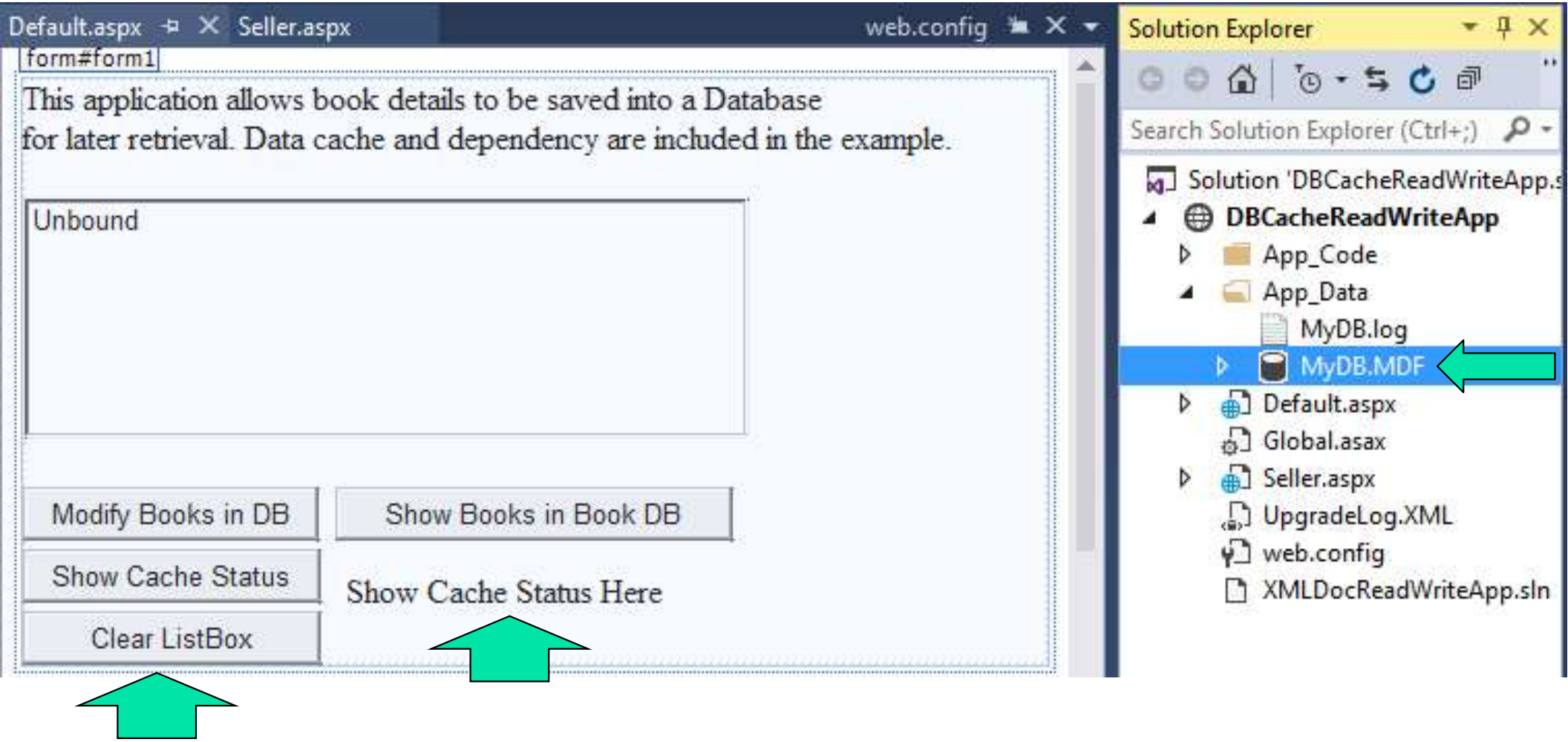
Clear ListBox

Read Text Section 5.5.2

Modify Book.xml

Bookstore Example: Saving to Database

This is example saves the books detail to a SQL database, instead of in a XML file.



Code Behind the Default.aspx (DB)

```
List<Book> objCache =
                    (List<Book>)System.Web.HttpRuntime.Cache.Get("BooksKey");
itemRemoved = false;
onRemove = new CacheItemRemovedCallback(this.RemovedCallback);
if (objCache == null)  {
    DBContextDataContextDataContext myDB = new
                        DBContextDataContextDataContext();
    IQueryable<Book> q = from b in myDB.Books select b;
    ListBox1.Items.Clear();
    foreach (Book child in q) {
        ListBox1.Items.Add(child.Title + "  " + child.Isbn + "  " + "$" + child.Price);
    }
    objCache = q.ToList<Book>();
    SqlCacheDependency dependency = new
            SqlCacheDependency(myDB.GetCommand(q) as SqlCommand);
    HttpContext.Current.Cache.Add("BooksKey", objCache, dependency,
    DateTime.MaxValue, TimeSpan.Zero, CacheItemPriority.Default,
                        new CacheItemRemovedCallback(onRemove));
}
```

Read data from DB and place into myDB

Query the object

Define dependency and add data into cache

Code Behind the Seller.aspx, Submit Books (DB)

```
string title2 = txtTitle2.Text;  // Book 2 info
string isbn2 = txtIsbn2.Text;
string sPrice2 = txtPrice2.Text;
Book book2 = new Book();
book2.Title = title2;
book2.Isbn = isbn2;
book2.Price = float.Parse(sPrice2);
myDB.Books.InsertOnSubmit(book2); // Save into book object
string title3 = txtTitle3.Text;  // Book 3 info
string isbn3 = txtIsbn3.Text;
string sPrice3 = txtPrice3.Text;
Book book3 = new Book();
book3.Title = title3;
book3.Isbn = isbn3;
book3.Price = float.Parse(sPrice3);
myDB.Books.InsertOnSubmit(book3); // Save into book object
myDB.SubmitChanges();             // Submit the all changes to DB
Cache.Remove("BooksKey");         // force removing cache
```

Submit into object

Submit into object

Submit into SQL database

What Data to Cache?
It is a part of your business logic!

- Data caching is useful not only for quick response time, but also for many other purposes, such as recommendation;
- Manufacture recommendation: For each product, the manufacture suggests a list of accessories:
  - Static, no caching required, but can recommend invalid items
- "Most Frequently Bought (MFB)" items to inspire customers to buy popular and relevant items;
- "Frequently Bought Together (FBT)" items: For each product a customer is viewing, show the list of FBT.
  - The list is dynamic and will be update frequently
  - Using social networking/collaborative model -- datamining

Caching vs. Recommendation

- As a part of the proactive intelligent business, many websites generate a recommendation list, which ranks a set of available choices based on certain given criteria and available set of inputs;
- Caching buffers the recently accessed information;
- They are different techniques, but they support each other:
  - Caching provides recently accessed information, which can be used as a criterion and as an input to recommendation;
  - Recommendation system generates important information, which can be used as a replacement policy for caching.

What is a Recommendation System?

Text Section 11.3.2

- A recommendation system is a subclass of information filtering system that predicts the rating, ranking, and preference, which a user would give to an item, or the probability that a user would buy the item.
- Many websites generate a recommendation list for a user when the user is viewing an item, which shows the available choices of alternatives or complementary items based on certain criteria and available set of inputs.
- The goal of recommendation is to increase the sale.

Recommendation in Amazon.com

www.amazon.com

Apple iPhone 14, 128GB, Midnight - Unlocked (Renewed)

Options: 3 sizes

4,021

3K+ bought in past month

$371.74 List Price: ~~$434.60~~

Display Size
6.1 inches

Memory
128 GB

See options

More Buying Choices
$320.86 (59+ used & new offers)

+4

How do you implement the "Frequently Bought Together" List?

Customers also bought

Based on products customers bought together

This item: Apple iPhone 14, 128GB, Midnight - Unlocked (Renewed)
4.1 4,021
-11% $385.00
~~$434.60~~

+

Sponsored
TOCOL 5 in 1 for iPhone 14 Case, with 2 Pack Screen Protector + 2 Pack Camera Lens Protector,...
4.5 5,378
$15.99
prime

+

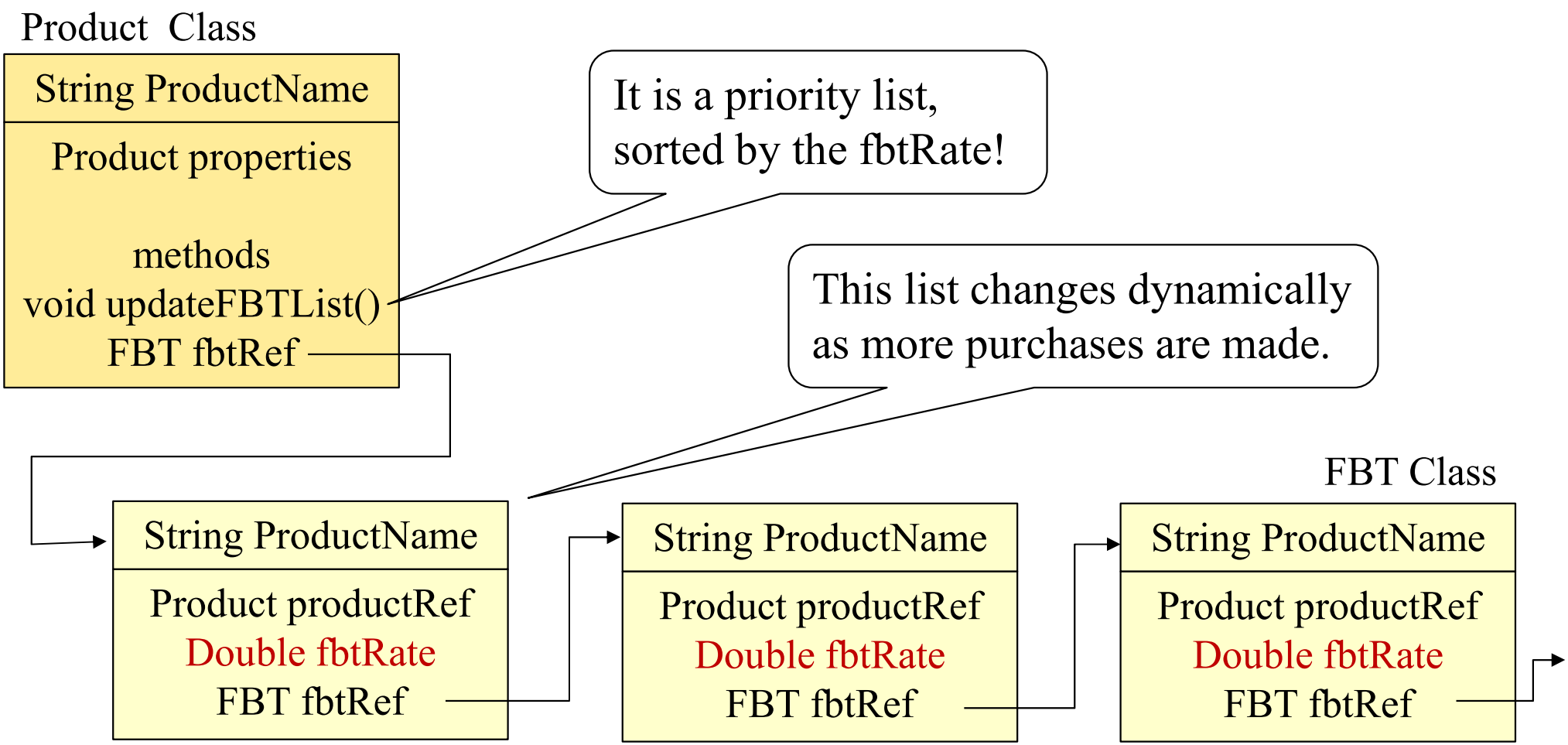
Sponsored
OtterBox iPhone 14 & iPhone 13 Commuter Series Case - Black, Slim & Tough, Pocket-Friendly,...
4.6 2,370
-25% $29.99
List: ~~$39.95~~
prime

Total price: $430.98

Add all 3 to Cart

These items are shipped from and sold by different sellers.
Show details

Simple Implementation Idea of the Frequently Bought Together Items



Search Engine vs. Recommendation System

Most criteria for a recommendation system also apply to a search engine. The main differences are

- Both generate a ranked list of information based on user input:
  - Search engine's input is explicit.
  - Recommendation system's input is implicit.
- Both try to generate the most relevant list to users
  - Search engine generates hot spot information + sponsored list
  - Recommendation system tries to avoid hot spot items, which can bring more benefit to the business.
- Information Sources:
  - Search engine uses public information mostly.
  - Recommendation system can use the internal database such as user profiles

Strategies of Recommendation

- *Social filtering or collaborative filtering*
  - Use the behaviors and preferences of collective users on the concerned items as the basis of recommendation;
- *Content-based filtering*
  - Use the attributes and semantics of the concerned items to recommend similar items;
- *User profile filtering*
  - Based on the age, gender, the historical behavior (past shopping list) of the user, etc., to recommend;
- *Situation-aware filtering*
  - Based on availability of the items, promotional information of the items, etc., to recommend.

Collaborative Filtering Spectrum

Collaborative Filtering (CF) Algorithms

Live data collected

Model-based

Math models based on past and current data

Memory-based

User-based

Item-based

Machine-Learning

- Training →Model
- Model-based
- Memory-based
- Model improvement

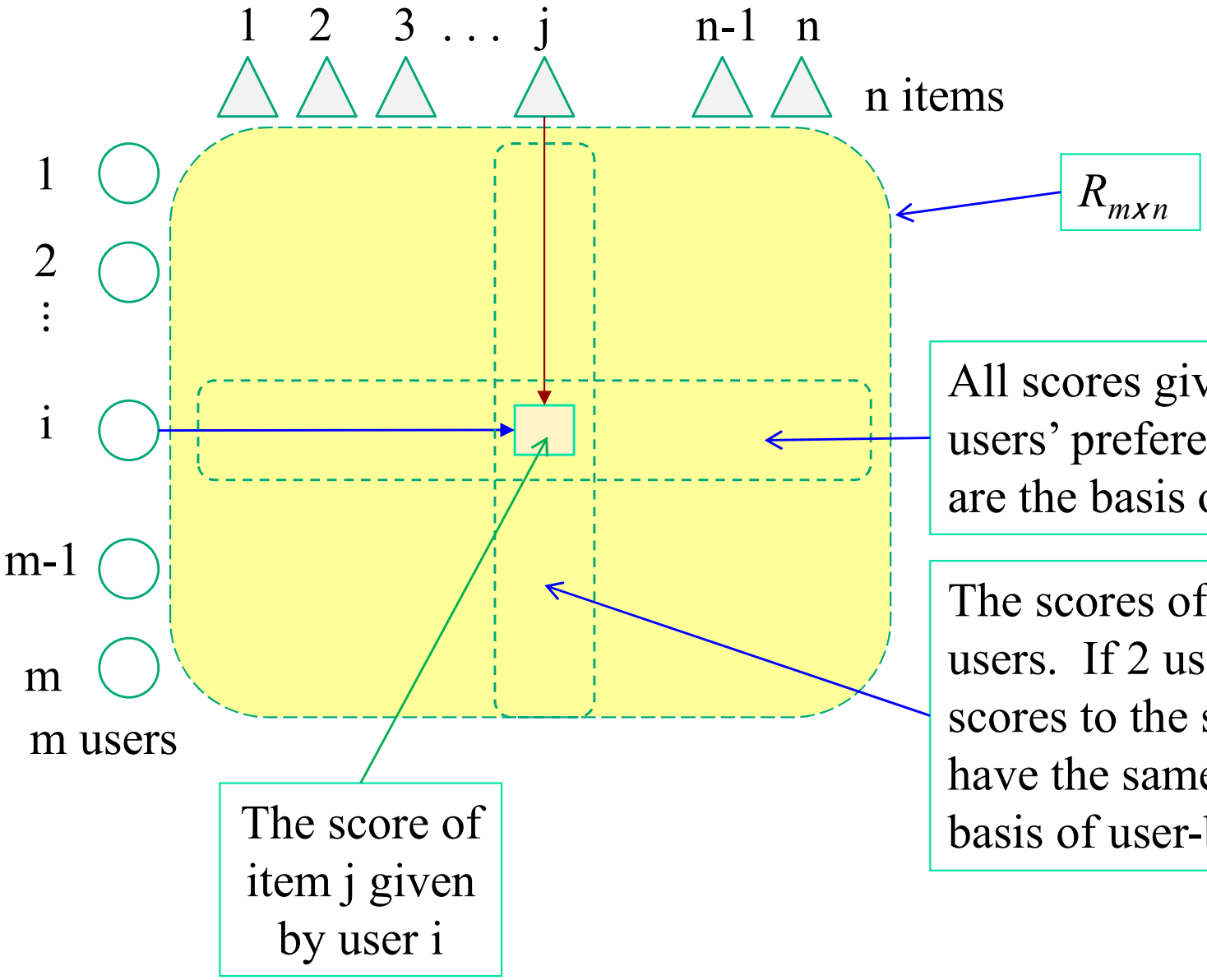
Ranking-oriented CF
(order in ranking)

Rating-oriented CF
(the numeric value)

Collaborative Filtering (CF)

- Main idea of CF is to group users into neighborhoods based on their product preferences and review scores
  - Model-based CF uses a mathematical, statistical, or learning model to analyze data and make predictions.
  - User-based CF estimates the unknown ratings of a target user based on the ratings given by *neighboring* users.
  - For item-based CF algorithms, item-item similarities are used to select a set of *neighboring* items that have been rated by the target user.

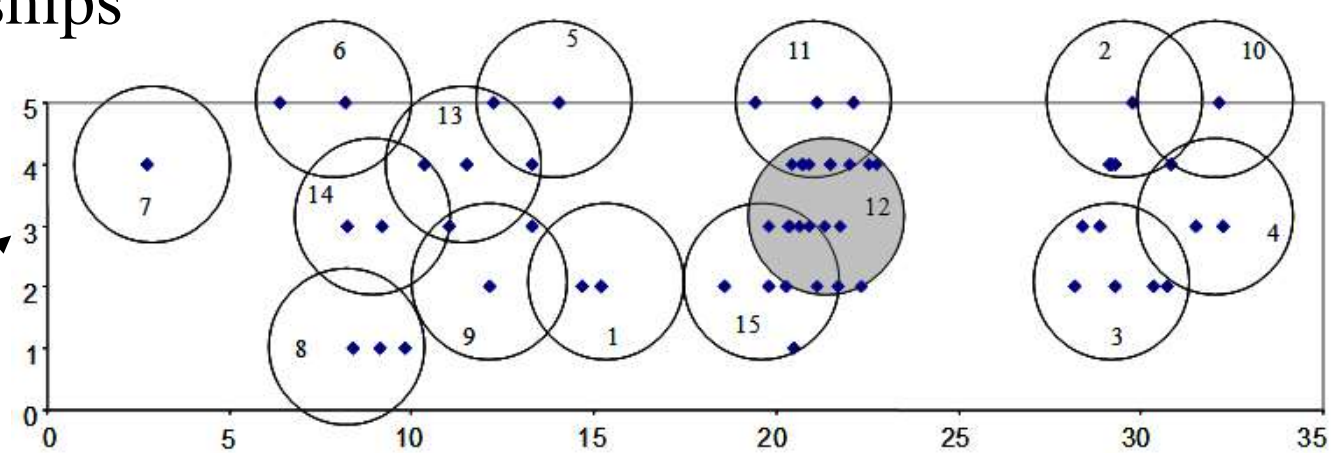
Product-User-Score Matrix for Recommendation



Rating-Oriented vs. Ranking-Oriented

- Both rating-oriented CF and ranking-oriented CF can be applied to memory-based or model-based.
- Rating-oriented CF algorithms group (cluster) the users based on users' historical rating scores on the items they review. If users give similar rating scores to the same items, they are considered *neighbors* in their judgment or taste.
- Ranking-oriented CF algorithms group the users based on the ranking order of items that they review, which ignores particular values of rating scores but maintains the ordered preference relationships between the items.

Apply clustering algorithm: Unsupervised learning, such as k-NN (k-nearest neighbors)



Criteria of Good Recommendation

- *Accuracy* represents the rate of relevancy between recommendation results and actual content, resulting in a high purchase rate.
- *Diversity* refers to a reasonable extension to the content the user is accessing.
- *Non-Hot spot Coverage*: This criterion has been generally admitted as a critical one. The recommendation should avoid hot topic data, which can bring more benefit to the site.
- *No "Cool Start"*: Cool start is a situation when the system cannot make a recommendation, as it has not gathered sufficient information yet. Social filtering can suffer from *cool start*.
- *Performance*: A system must provide real-time recommendation with live interaction with users.

Example: Situation Aware Mobile Recommendation

- A typical scenario at, e.g., Costco store or Amazon Go store: People used to their coupon books to search the items in store.
- Installing a mobile "app" on your smart phone, with the support from store backend Web application:
  - Pop up the item in the coupon book as you are near the location of the item;
  - Recommend the items you need to buy, based on the information what you bought before and how frequent you buy the items before, for example, your water filter.
  - Recommend the best-sold items in the last 7 days
  - Recommend the most valuable item to buy (non-hot spot)
  - URL-based recommendation: read the URL history, analyse the items in the URL sites and recommend similar items.

Legal and Ethics Issues in Caching/Recommendation

- Technologies are making more and more information available.
- Balance between information privacy and openness
- Developers need to be aware of legal and ethics issues in data caching and recommendation. For examples
  - ASU has the organizational access to IEEE digital library (publications). Caching data and making them available to non-ASU users would violate the copyright law.
  - It is illegal to cache or recommend illegal data. It is the responsibility of the business and developer to detect and recognize illegal data.

Legal and Ethics Issues in Caching/Recommendation

- Developers need to be aware of legal and ethics issues in recommendation. For examples
  - Is it ethical to use user's profile or URL history to make recommendation?
  - Is it legal or ethical to explore personal emails and social networking sites, and phone conversations to make recommendation?
  - Individual sources are open/public information, should the combination and integration of the sources still be open/public?

More lectures later on

- Proactive artificial intelligent services and applications
- On Demand e-business

- In the Unit on Big Data Process and Artificial Intelligence

On Demand e-business with AI-based real-time reconfigure-ation, resilient, & automated evolution

Web 5.0 with proactive artificial intelligent services and applications*